

Nettoyage et préparation de données

Pour la préparation des données, nous avons utilisé dans un premier temps **R** afin d'effectuer les opérations de nettoyage et de transformation des données (gestion des valeurs manquantes, formats de dates, contrôle de la qualité des données). Nous avons ensuite reproduit ces traitements sous **Python** afin de vérifier la cohérence des résultats obtenus. Après cette phase de validation, nous avons choisi de poursuivre l'ensemble du projet sous Python afin de centraliser les traitements de données, les analyses, les modèles de machine learning et le développement de l'API au sein d'un même environnement.

Langage R :

1. Importation des données et bibliothèques

Nous avons commencé par charger les bibliothèques nécessaires à l'analyse :

- `readxl` pour la lecture des fichiers
- `dplyr` pour la manipulation des données
- `tidyr` pour la structuration des données

```
#-----Lire les données-----  
library(readxl) #library pour la lecture des fichiers  
library(dplyr)  
library(tidyr)
```

Ensuite, chaque fichier CSV a été importé dans R à l'aide de la fonction `read.csv()`.

```
#Importation de nos données:  
actifs_si <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/actifs_si.csv",  
                    sep = ",",  
                    header = TRUE)  
View(actifs_si)
```

2. Analyse exploratoire des données

Pour chaque jeu de données, nous avons effectué les étapes suivantes :

- Visualisation des données avec `View()`
- Vérification des dimensions avec `dim()`
- Détection des valeurs manquantes avec `colSums(is.na(...))`
- Identification des doublons avec `duplicated()` et `sum(duplicated(...))`
- Analyse de la structure avec `str()`

```
#chercher les lignes dupliquées
sum(duplicated(actifs_si))
dim(actifs_si) #Le nombre de lignes et colonnes
```

3. Nettoyage des données

Les opérations de nettoyage suivantes ont été appliquées :

- Suppression des valeurs manquantes avec `na.omit()`
- Suppression des doublons (dans certains fichiers avec `distinct()`)
- Conversion des types de données :
 - Dates converties avec `as.Date()`
 - Horodatages convertis avec `as.POSIXct()`
 - Variables numériques converties avec `as.numeric()`

```
#changer le type de la colonne date_detection en format date
cas_fraude_confirmes$date_detection <- as.Date(cas_fraude_confirmes$date_detection)
str(cas_fraude_confirmes) #Vérification
class(cas_fraude_confirmes$date_detection)
```

4. Traitement par fichier

♦ actifs_si

- Importation et vérification des données
- Détection des NA et doublons
- Exportation du fichier nettoyé

```
#----- fichier actif_si-----

#Importation de nos données:
actifs_si <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/actifs_si.csv",
                    sep = ",",
                    header = TRUE)

View(actifs_si)
#s'il y'a des lignes vides ou non
colSums(is.na(actifs_si))
#chercher les lignes dupliquées
sum(duplicated(actifs_si))
dim(actifs_si) #Le nombre de lignes et colonnes

# Enregistrer le fichier actifs_si dans ce chemin en Format csv
write.csv(actifs_si,file = "C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees nettoyees/actifs_si.csv")
```

♦ cas_fraude_confirmes

- Conversion de la variable `date_detection` en format date
- Vérification de la structure des données

```
#----- fichier cas_fraude_confirmes -----
#Importation de nos données:
cas_fraude_confirmes <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/cas_fraude_confirmes.csv",
                                sep = ",",
                                header = TRUE)
View(cas_fraude_confirmes)
dim(cas_fraude_confirmes) #Le nombre de lignes et colonnes

#s'il y'a des lignes vides ou non
colSums(is.na(cas_fraude_confirmes))
#chercher les lignes dupliquées
sum(duplicated(cas_fraude_confirmes))
#savoir le type de chaque colonne
str(cas_fraude_confirmes)

#changer le type de la colonne date_detection en format date
cas_fraude_confirmes$date_detection <- as.Date(cas_fraude_confirmes$date_detection)
str(cas_fraude_confirmes) #Vérification
class(cas_fraude_confirmes$date_detection)

# Enregistrer le fichier cas_fraude_confirmes dans ce chemin en Format csv
write.csv(cas_fraude_confirmes, file = "C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees nettoyees/cas_fraude_confirmes.csv")
```

◆ clients

- Suppression des valeurs manquantes
- Conversion de **date_entree** et **surface_m2**
- Analyse de la structure

```
#----- fichier clients -----
#Importation de nos données:
clients <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/clients.csv",
                    sep = ",",
                    header = TRUE)
View(clients)
dim(clients) #Le nombre de lignes et colonnes avant le nettoyage
#s'il y'a des lignes vides ou non
colSums(is.na(clients))
clients <- na.omit(clients) #supprimer les NA
#chercher les lignes dupliquées
sum(duplicated(clients))

dim(clients) #Le nombre de lignes et colonnes apres le nettoyage

#savoir le type de chaque colonne
str(clients)

#changer le type de la colonne date_entree en format date
clients$date_entree <- as.Date(clients$date_entree)
#changer le type de la colonne surface_m2 en float (numeric)
clients$surface_m2 <- as.numeric(clients$surface_m2)
str(clients) #Vérification

# Enregistrer le fichier actifs_si dans ce chemin en Format csv
write.csv(clients, file = "C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees nettoyees/clients.csv")
```

◆ compteurs

- Nettoyage des NA
- Conversion de **date_pose** en format date

```
#----- fichier compteurs-----
#Importation de nos données:
compteurs <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/compteurs.csv",
  sep = ",",
  header = TRUE)
View(compteurs)
dim(compteurs) #Le nombre de lignes et colonnes avant le nettoyage
#s'il y'a des lignes vides ou non
colSums(is.na(compteurs))
compteurs <- na.omit(compteurs) #supprimer les NA

#chercher les lignes dupliquées
sum(duplicated(compteurs))

dim(compteurs) #Le nombre de lignes et colonnes apres le nettoyage

#savoir le type de chaque colonne
str(compteurs)

#changer le type de la colonne date_pose en format date
compteurs$date_pose <- as.Date(compteurs$date_pose)
str(compteurs) #Vérification

# Enregistrer le fichier actifs si dans ce chemin en Format csv
write.csv(compteurs, file = "C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees nettoyees/compteurs.csv")
```

◆ incidents_reseau

- Suppression des valeurs manquantes
- Conversion de **date_debut** en format datetime (**POSIXct**)

```
#----- fichier incidents_reseau-----
#Importation de nos données:
incidents_reseau <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/incidents_reseau.csv",
  sep = ",",
  header = TRUE)
View(incidents_reseau)
dim(incidents_reseau) #Le nombre de lignes et colonnes avant le nettoyage
#s'il y'a des lignes vides ou non
colSums(is.na(incidents_reseau))
incidents_reseau <- na.omit(incidents_reseau) #supprimer les NA
#chercher les lignes dupliquées
sum(duplicated(incidents_reseau))

dim(incidents_reseau) #Le nombre de lignes et colonnes apres le nettoyage

#savoir le type de chaque colonne
str(incidents_reseau)

#changer le type de la colonne date_debut en format date
incidents_reseau$date_debut <- as.POSIXct(incidents_reseau$date_debut, format = "%Y-%m-%d %H:%M")
class(incidents_reseau$date_debut)
str(incidents_reseau$date_debut)
format(incidents_reseau$date_debut, "%Y-%m-%d %H:%M")
str(incidents_reseau) #Vérification

# Enregistrer le fichier actifs si dans ce chemin en Format csv
write.csv(incidents_reseau, file = "C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees nettoyees/incidents_reseau.csv")
```

◆ journaux_securite

- Conversion de **horodatage** en format datetime

```
#----- fichier journaux_securite-----
#Importation de nos données:
journaux_securite <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/journaux_securite.csv",
                             sep = ",",
                             header = TRUE)
View(journaux_securite)
dim(journaux_securite) #Le nombre de lignes et colonnes avant le nettoyage
#s'il y'a des lignes vides ou non
colSums(is.na(journaux_securite))
journaux_securite <- na.omit(journaux_securite) #supprimer les NA
#chercher les lignes dupliquées
sum(duplicated(journaux_securite))

dim(journaux_securite) #Le nombre de lignes et colonnes apres le nettoyage

#savoir le type de chaque colonne
str(journaux_securite)

#changer le type de la colonne date_pose en format date
journaux_securite$horodatage <- as.POSIXct(journaux_securite$horodatage, format = "%Y-%m-%d %H:%M:%S")

class(journaux_securite$horodatage)
str(journaux_securite$horodatage)
format(journaux_securite$horodatage, "%Y-%m-%d %H:%M:%S")
str(journaux_securite) #Vérification

# Enregistrer le fichier actifs si dans ce chemin en Format csv
write.csv(journaux_securite,file = "C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees nettoyees/journaux_securite.csv")
```

♦ meteo

- Conversion de **date** en format date

```
#----- fichier meteo-----
#Importation de nos données:
meteo <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/meteo.csv",
                  sep = ",",
                  header = TRUE)
View(meteo)
dim(meteo) #Le nombre de lignes et colonnes avant le nettoyage
#s'il y'a des lignes vides ou non
colSums(is.na(meteo))
meteo <- na.omit(meteo) #supprimer les NA
#chercher les lignes dupliquées
sum(duplicated(meteo))

dim(meteo) #Le nombre de lignes et colonnes apres le nettoyage

#savoir le type de chaque colonne
str(meteo)

#changer le type de la colonne date en format date
meteo$date <- as.Date(meteo$date)
str(meteo) #Vérification

# Enregistrer le fichier actifs si dans ce chemin en Format csv
write.csv(meteo,file = "C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees nettoyees/meteo.csv")
```

♦ releves_consommation

- Suppression des NA et doublons
- Conversion de **date** en format date

```
#----- fichier releves_consommation-----

#Importation de nos données:
releves_consommation <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/releves_consommation.csv",
                                sep = ",",
                                header = TRUE)
View(releves_consommation)
dim(releves_consommation) #Le nombre de lignes et colonnes avant le nettoyage
#s'il y'a des lignes vides ou non
colSums(is.na(releves_consommation))
releves_consommation <- na.omit(releves_consommation) #supprimer les NA
#chercher les lignes dupliquées
sum(duplicated(releves_consommation))

# Supprimer les doublons
releves_consommation <- distinct(releves_consommation)

dim(releves_consommation) #Le nombre de lignes et colonnes apres le nettoyage

#savoir le type de chaque colonne
str(releves_consommation)

#changer le type de la colonne date en format date
releves_consommation$date <- as.Date(releves_consommation$date)
str(releves_consommation) #Vérification

# Enregistrer le fichier actifs si dans ce chemin en Format csv
write.csv(releves_consommation, file = "C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees nettoyees/releves_consommation.csv")
```

◆ réclamations

- Suppression des valeurs manquantes
- Conversion de **date** en format date

```
#----- fichier reclamations-----

#Importation de nos données:
reclamations <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/reclamations.csv",
                          sep = ",",
                          header = TRUE)
View(reclamations)
dim(reclamations) #Le nombre de lignes et colonnes avant le nettoyage
#s'il y'a des lignes vides ou non
colSums(is.na(reclamations))
reclamations <- na.omit(reclamations) #supprimer les NA
#chercher les lignes dupliquées
sum(duplicated(reclamations))

#changer le type de la colonne date en format date
reclamations$date <- as.Date(reclamations$date)
str(reclamations) #Vérification

#Importation de nos données:
write.csv(reclamations, file = "C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees nettoyees/reclamations.csv")
```

◆ releves_horaires_echantillon

- Suppression des NA
- Conversion de **date** et **horodatage** en formats adaptés

```

#Importation de nos données:
relevés_horaires_échantillon <- read.csv("C:/Users/Lenovo/Desktop/school/ESIC/examen final/donnees/donnees/relevés_horaires_e
      sep = ",",
      header = TRUE)

View(relevés_horaires_échantillon)
dim(relevés_horaires_échantillon) #Le nombre de lignes et colonnes avant le nettoyage
#s'il y'a des lignes vides ou non
colSums(is.na(relevés_horaires_échantillon))
relevés_horaires_échantillon <- na.omit(relevés_horaires_échantillon) #supprimer les NA
#chercher les lignes dupliquées
sum(duplicated(relevés_horaires_échantillon))

dim(relevés_horaires_échantillon) #Le nombre de lignes et colonnes après le nettoyage

#changer le type de la colonne date en format date
relevés_horaires_échantillon$date <- as.Date(relevés_horaires_échantillon$date)
str(relevés_horaires_échantillon) #Vérification

#changer le type de la colonne horodatage en format date
relevés_horaires_échantillon$horodatage <- as.POSIXct(relevés_horaires_échantillon$horodatage, format = "%Y-%m-%d %H:%M:%S")

class(relevés_horaires_échantillon$horodatage)
str(relevés_horaires_échantillon$horodatage)
format(relevés_horaires_échantillon$horodatage, "%Y-%m-%d %H:%M:%S")
str(relevés_horaires_échantillon) #Vérification

```

5. Sauvegarde des données nettoyées

Enfin, chaque dataset nettoyé a été sauvegardé au format CSV dans un dossier dédié afin d'être utilisé pour les analyses ultérieures.

Python :

En complément du travail réalisé sous R, nous avons également utilisé Python afin de reproduire les mêmes étapes de préparation et de nettoyage des données.

L'objectif de cette démarche était de valider et vérifier la cohérence des résultats obtenus sous R, notamment concernant la détection des valeurs manquantes, la suppression des doublons et la conversion des types de données.

Ainsi, Python a servi comme un outil de contrôle et de confirmation, permettant de s'assurer que les opérations de nettoyage étaient correctes et que les données finales étaient fiables pour les analyses futures.